

บทที่ 3

ทฤษฎีและความรู้พื้นฐานเกี่ยวกับ Load Balancing

บน BEA WebLogic 7.0

แนะนำเว็บโลจิกโหลดบาลานซิง (WebLogic Load Balancing)

ทุกๆ WebLogic เซิร์ฟเวอร์ ใน คลัสเตอร์ ต้องการถูกใช้งานอย่างเต็มที่เพื่อรองรับงานปริมาณมาก จึงมีการใช้วิธีโหลดบาลานซิงมาใช้ในการแบ่งงานระหว่างโฮส ถ้าโฮสไหนเกิดเสียโหลดบาลานซิงจะรับประกันว่ามีโฮสอื่นมารองรับการทำงานแทนโฮสนั้นได้ เราสามารถตั้งค่าโหลดบาลานซิงต่างๆได้เมื่อติดตั้ง คลัสเตอร์ โดยจะมีวิธีการทำได้ 3 แบบดังนี้

1. Round-robin(default)
2. Weight based
3. Random

Round-Robin โหลดบาลานซิง

Round-Robin จะทำการวนโฮสใน คลัสเตอร์ ไปเรื่อยในการทำงาน ตัวโฮสที่ทำคลัสเตอร์ จะรับข้อมูลมาแล้วส่งไปยังเซิร์ฟเวอร์ ที่ว่างตามลำดับการเข้ามาของข้อมูลวนไปเรื่อยๆ การใช้วิธีนี้เหมาะกับเครื่องที่มีประสิทธิภาพพอๆกัน

การทำ Round-Robin นั้นง่าย ประหยัด และสามารถคาดการณ์ได้ แต่ข้อเสียหลักของมันจะเกิดเมื่อมีเซิร์ฟเวอร์ หนึ่งช้ากว่าตัวอื่นจะทำให้ข้อมูลที่เข้ามาต้องรอเซิร์ฟเวอร์ นั้นทำให้ขบวนการทั้งหมดช้าลงได้

Weight-Based โหลดบาลานซิง

ดีกว่า Round-robin โดยที่ weight-based จะนำค่าน้ำหนักของแต่ละ เซิร์ฟเวอร์ มาคิดด้วย เซิร์ฟเวอร์แต่ละตัวจะมีค่าน้ำหนักตั้งแต่ 1-100 โดยอัตราส่วนนี้จะขึ้นกับ เซิร์ฟเวอร์ อื่นด้วย เช่น มี เซิร์ฟเวอร์ เครื่องหนึ่ง ตั้งค่าไว้ที่ 50 นอกนั้นเป็น 100 หก เซิร์ฟเวอร์ ตัวนั้นก็จะแบ่งงานไป 50 ของอัตราส่วนน้ำหนักของเซิร์ฟเวอร์ ที่เหลือ วิธีนี้จะใช้ได้กับเฉพาะ ออบเจกต์ และไม่ใช่ session states

Random โหลดบาลานซิง

วิธีนี้จะเลือกโฮสโดยวิธีการสุ่มจะใช้ได้เฉพาะกับออบเจกต์ โดยมีแนวโน้มที่จะกระจายข้อมูลที่เข้ามาอย่างเท่ากันระหว่างโฮส วิธีนี้ควรใช้ใน คลัสเตอร์ ที่แต่ละโฮสมีบริการต่างๆเหมือนกันและมีประสิทธิภาพเหมือนกัน ข้อเสียของวิธีนี้คือการเสียเวลาในการสร้างตัวเลขในการสุ่มขึ้นมา

โหนดบาลานซ์สำหรับ HTTP Session

โหนดบาลานซ์ สำหรับ Servlets และ JSP HTTP session states สามารถทำโดยใช้วิธีดังนี้

-Built-in load-balancing capabilities of WebLogic proxy plug-ins การใช้ proxy plug-ins จะติดตั้งบนเว็บเซิร์ฟเวอร์ยี่ห้อใดก็ได้ แต่จะใช้ได้เฉพาะกับวิธี round-robin เท่านั้นในการกระจายคำร้องขอที่เป็น servlets และ JSP

-Load-balancing hardware เป็นการใช้โหนดบาลานซ์ที่เป็นฮาร์ดแวร์เข้ามาช่วยโดยจะสามารถใช้วิธีการทำโหนดบาลานซ์ได้หลากหลาย

การรองรับเพลโอเวอร์

เว็บโลจิกเซิร์ฟเวอร์ สามารถตรวจสอบความเสียหายของเซิร์ฟเวอร์ ได้จากวิธีต่อไปนี้

-Socket connection แบบต่อตรงกับ เซิร์ฟเวอร์ (Peer-to-peer)

-ข้อความ heartbeat

เซิร์ฟเวอร์ จะดูแลการใช้ IP socket ระหว่าง peer เซิร์ฟเวอร์ ถ้า เซิร์ฟเวอร์ ต่อเข้ากันแบบ peer เริ่มส่งข้อมูลโดยใช้ socket และเมื่อมีการปิดลงของการติดต่อ socket ก็รู้ว่า เซิร์ฟเวอร์ นั้นเสีย และงานบริการต่างที่ เซิร์ฟเวอร์ นั้นทำจะถูกลบออกจาก JNDI naming tree การตรวจสอบยังสามารถใช้ heartbeat ในการตรวจสอบรายละเอียดของ heartbeat และ การติดต่อแบบsocket จะอธิบายในหัวข้อต่อไป

การสื่อสารในคลัสเตอร์

เว็บโลจิกเซิร์ฟเวอร์ จะติดต่อกันระหว่างโฮสโดยใช้วิธีต่อไปนี้

-IP multicast

-IP socket

การสื่อสารแบบ IP Multicast-One-to-Many

เว็บโลจิกเซิร์ฟเวอร์ ใช้ IP multicast ในการตรวจสอบความล้มเหลวของ เซิร์ฟเวอร์ และดูแล Cluster-wide JNDI tree IP multicast เป็นการ broadcast โดยใช้ IP address ตั้งแต่ 239.255.255.255 ถึง 224.0.0.0 IP multicast จะไม่รับประกันความสูญหายของข้อความ

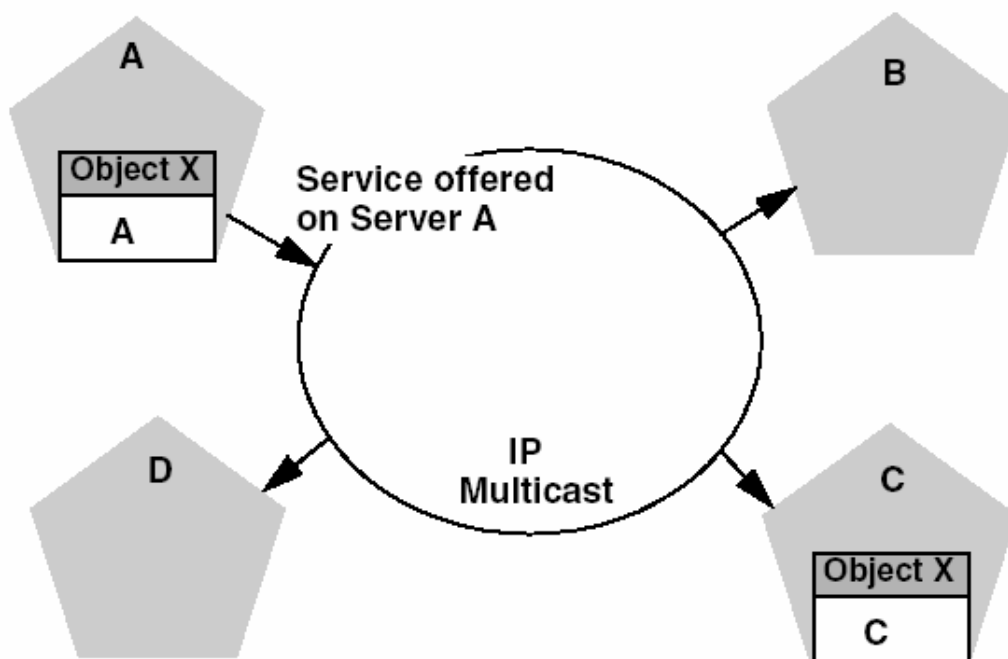
เว็บโลจิกเซิร์ฟเวอร์ ใช้ IP multicast ในการทำงานต่อไปนี้

-Cluster-wide JNDI updates

-Cluster heartbeats

Cluster-Wide JNDI Updates

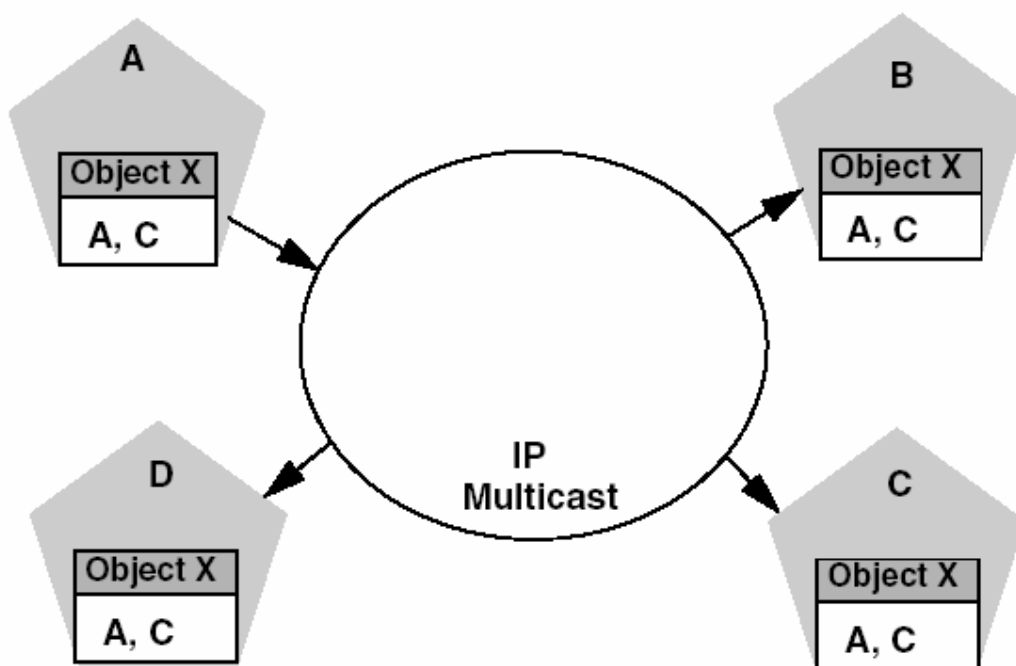
ทุกเซิร์ฟเวอร์ ใช้ IP multicast ในการประกาศว่าสามารถใช้ขอบเขตได้ มีการติดตั้งเพิ่มหรือมีการเอาตัวไหนออกบ้างเซิร์ฟเวอร์ จะตรวจสอบการประกาศแล้วปรับปรุง JNDI tree ใหม่



รูปที่ 3-1 การอัปเดต JNDI tree

การอัปเดต JNDI จะเกิดขึ้นได้ใน 2 กรณี

1. ถ้าการทำงานคลัสเตอร์ยังไม่ต่อเข้ากับ JNDI tree เซิร์ฟเวอร์จะต่อ replica-aware stub กับ โคลด (local) ออบเจ็กต์ของเซิร์ฟเวอร์ก่อน เช่น เซิร์ฟเวอร์ A จะอัปเดตออบเจ็กต์ X โดย เซิร์ฟเวอร์ที่เหลือก็จะทำเหมือนกันโดยเซิร์ฟเวอร์ A และ เซิร์ฟเวอร์ C จะมีออบเจ็กต์ X เหมือนกัน
2. ในกรณีที่คลัสเตอร์ทำงานแล้วจะมีการประกาศออบเจ็กต์ที่ตนเองมีให้เซิร์ฟเวอร์อื่นทราบแล้วทำการอัปเดตข้อมูลใหม่โดยเมื่อรับทราบครบทุกเซิร์ฟเวอร์แล้วจะได้ดังรูปที่ 3-2



รูปที่ 3-2 การอัปเดต JNDI tree ที่สมบูรณ์

Cluster Heartbeats

เว็บโลจิกเซิร์ฟเวอร์ ใช้การประกาศข้อความ Heartbeat ที่บอกถึงความคงอยู่ของ เซิร์ฟเวอร์ นั้น ในคลัสเตอร์ทุกเซิร์ฟเวอร์ รับข้อความ heartbeat เพื่อนำมาช่วยวิเคราะห์ว่าเซิร์ฟเวอร์เสียเมื่อไหร่

IP multicast สามารถเชื่อถือได้เมื่อส่งบนชั้นเน็ตเดียวกัน อย่างไรก็ตามถ้าต้องการแยกเซิร์ฟเวอร์ไว้ในที่อื่น จะมีโอกาสที่จะต้องผ่านหลายๆชั้นเน็ต เพื่อรับประกันว่า IP multicast ถูกส่งอย่างถูกต้อง เน็ตเวิร์คต้องเป็นตามความต้องการดังนี้

1. ต้องสนับสนุน IP multicast packet propagation ทุกเราท์เตอร์ต้องตั้งค่าในการเผยแพร่ข้อความ multicast ไปยัง เซิร์ฟเวอร์ นั้น
2. ความล่าช้าของการส่งต้องน้อยเพื่อรับประกันว่าสามารถส่งข้อความ multicast ถึงได้ใน 200 ถึง 300 มิลลิวินาที
3. ค่า TTL (time to live) ต้องสูงเพียงพอที่จะรับประกันว่าเราท์เตอร์จะไม่ทิ้งแพคเกจ ก่อนที่จะถึงปลายทาง สามารถตั้งค่าได้ใน *config.xml* ที่ค่า MulticastTTL ใน คลัสเตอร์ element

<คลัสเตอร์

...

MulticastTTL="2"

/>

หรือใช้เว็บโลจิกคอนโซลในการตั้งค่า ค่า MulticastTTL เป็น 2 บอกว่าสามารถส่งผ่าน 2 เราท์เตอร์ได้ก่อนที่จะถูกทิ้ง

IP Socket-Peer-to-Peer Communication

เว็บโลจิกเซิร์ฟเวอร์ ใช้ IP socket กับการทำงานต่อไปนี้

- เพื่อเข้าถึงออบเจกต์ที่ไม่ได้ทำคลัสเตอร์จากระยะไกล

- เพื่อคัดลอก HTTP session states และ stateful session EJB states จากเซิร์ฟเวอร์หลักไปยังเซิร์ฟเวอร์ที่สอง เพื่อความสามารถในการใช้งานสูงขึ้นและรองรับโหลดโอเวอร์

- เพื่อเข้าถึงออบเจกต์ที่ทำคลัสเตอร์จากระยะไกล

ปัจจัยสองอย่างที่กระทบประสิทธิภาพของการติดต่อแบบ socket คือ

- ใช้ native หรือ pure Java socket reader implementationหรือไม่

- ตั้งค่า socket reader threads ใน pure Java socket reader หรือไม่

Native socket reader implementation ให้ประสิทธิภาพดีที่สุดสำหรับการใช้งาน socket มากๆ Java socket reader รวมทุก socket แม้จะมีบางอันที่ไม่มีข้อมูล ส่วน native หรือ pure ตัว socket reader จะมีประสิทธิภาพกว่าเพราะจะรวบรวมเฉพาะ socket ที่ใช้ได้จริงและจะถูกบอกทันทีที่ socket ใช้งานได้

การติดต่อระหว่างผู้ใช้กับคลัสเตอร์

ผู้ใช้เข้าถึงออบเจกต์หรือเซอร์วิสของเว็บโลจิกเซิร์ฟเวอร์ โดยใช้ JNDI-compliant

naming เซอร์วิส ซึ่งเก็บรายการเซอร์วิสต่างๆที่ให้บริการ จัดการในรูปแบบ tree ตัวเซิร์ฟเวอร์เองก็ใช้ JNDI tree เพื่อเก็บเซอร์วิสบนเซิร์ฟเวอร์ เพื่อใช้ในการแสดงให้เซิร์ฟเวอร์อื่นในคลัสเตอร์เห็นว่ามิเซอร์วิส อะไรบ้างและมีเซอร์วิสอะไรที่ เซิร์ฟเวอร์อื่นมีบ้าง

แต่ละเซิร์ฟเวอร์สร้างและดูแลข้อมูลของ local JNDI tree ในตอนเริ่มแรกเซิร์ฟเวอร์จะใช้ local JNDI tree ก่อน แล้วส่งข้อมูลออบเจกต์ที่มีไปให้ทุกเซิร์ฟเวอร์ในคลัสเตอร์โดยใช้ multicast address แล้ว เซิร์ฟเวอร์อื่นก็จะทำการ update JNDI tree

การเกิดความขัดแย้งใน JNDI Naming

การเกิดความขัดแย้งใน JNDI Naming จะเกิดเมื่อเซิร์ฟเวอร์พยายามใช้ชื่อ JNDI ของเซิร์ฟเวอร์ที่ไม่ใช่คลัสเตอร์ที่มีชื่อซ้ำกับที่มีใน JNDI ของเซิร์ฟเวอร์นั้น ความขัดแย้งใน Cluster-level JNDIเกิดเมื่อเซิร์ฟเวอร์พยายามใช้ชื่อออบเจกต์ของเซิร์ฟเวอร์ที่ไม่ใช่คลัสเตอร์ ที่มีอยู่แล้วใน JNDI tree การจัดการความสับสนต่างๆทำได้ดังนี้

ถ้าสองเซิร์ฟเวอร์ในคลัสเตอร์พยายามใช้ ออบเจกต์ต่างกันโดยใช้ชื่อเดียวกัน ทั้งสองเซิร์ฟเวอร์ จะสามารถทำได้เฉพาะสองเซิร์ฟเวอร์นั้น อย่างไรก็ตามแต่ละเซิร์ฟเวอร์นั้นจะปฏิเสธการเก็บชื่อซ้ำลงบน JNDI tree การสับสนแบบนี้จะอยู่นานกว่าหนึ่งในสองเซิร์ฟเวอร์นั้นปิดลงหรือหนึ่งในนั้นปิด ออบเจกต์ ตัวนั้น

J2EE คลัสเตอร์ริง

J2EE สามารถให้บริการ คลัสเตอร์ริง ได้หลายทาง และ เว็บโลจิกเซิร์ฟเวอร์ ก็สามารถแบ่งการทำงานของ คลัสเตอร์ริง ได้ดังนี้

- JSP/servlets คลัสเตอร์ริง
- ออบเจ็ค คลัสเตอร์ริง
- Java Message เซอร์วิส (JMS) เซิร์ฟเวอร์ คลัสเตอร์ริง

HTTP JSP/Servlets คลัสเตอร์ริง

เว็บโลจิกเซิร์ฟเวอร์สามารถรองรับคลัสเตอร์ริงบนJSP และ servlet โดยการทำสำเนาข้อมูลของผู้ใช้ที่ทำการติดต่อกับเซิร์ฟเวอร์ผ่านทาง JSP และ servlets และเพื่อแน่ใจว่าการทำงานของเซสชันมีจริง สถานะของเซสชันต้องถูกป้องกัน และสถานะนี้สามารถถูกป้องกันได้หลายทางคือ

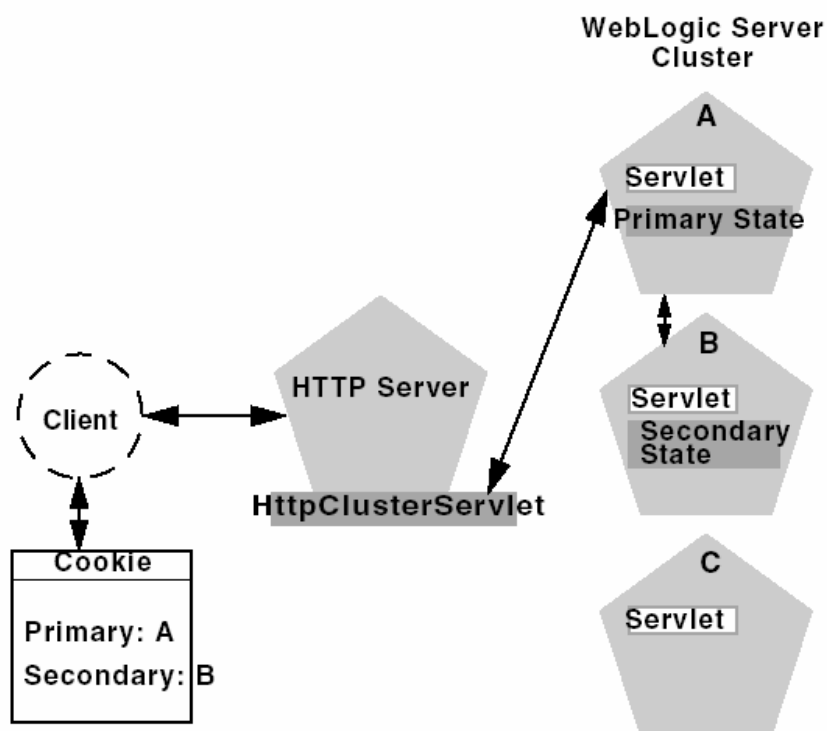
- In-memory replication
- File-based persistence
- JDBC-based persistence

In-Memory Replication

นี่เป็นการทำงานแบบทำสำเนาของสถานะของเซสชันจากเซิร์ฟเวอร์นี้ไปยังอีกเซิร์ฟเวอร์หนึ่ง เมื่อไคน์ที่ติดต่อยัง เว็บโลจิก เซิร์ฟเวอร์ และทำงานเกี่ยวกับ JSP / servlets เซสชันถูกสร้างขึ้นบนเซิร์ฟเวอร์ที่ไคน์ที่ติดต่อเข้ามาดังรูปที่ 3-3 สถานะนี้เรียกว่า primary session state ส่วนสถานะที่ถูกสำเนาไปยังเครื่องต่างๆ ในคลัสเตอร์เราเรียกว่า secondary session state ดังนั้น ถ้าเซิร์ฟเวอร์ที่ต้องตอบกลับไคน์เอนท์นั้นใช้งานไม่ได้ ก็จะมีการป้องกันการเฟลโอเวอร์โดยการให้เซิร์ฟเวอร์ตัวอื่นที่ได้ secondary session state ทำการ เฟลโอเวอร์ โดยอัตโนมัติ

และเพื่อเพิ่มประสิทธิภาพในการใช้ in-memory replication กับ HTTP session state เว็บโลจิกเซิร์ฟเวอร์ ต้องรองรับการทำงานดังนี้อีกด้วยคือ

- ผ่านทางโหนดบาลานซ์ฮาร์ดแวร์
- ผ่านทางกลุ่มของเว็บเซิร์ฟเวอร์โดยใช้เว็บโลจิกพรอกซี่เข้ามาช่วยในการติดต่อ



รูปที่ 3-3 การสำรองเซสชันของผู้ใช้

ความต้องการของโหนดบาลานเซอร์

โหนดบาลานซ์ฮาร์ดแวร์ต้องรองรับการทำงานของ in-memory replication ดังนี้คือ

- การเข้ารหัส SSL
- สามารถรองรับคูกี้ก็ได้

เว็บโลจิก เซิร์ฟเวอร์ ต้องรองรับการเขียนคูกี้ผ่านโหนดบาลานเซอร์ไปที่เครื่องไคลเอนต์ และ บาลานเซอร์ทำการควบคุมเครื่องไคลเอนต์ผ่านทางวิธีการเขียนคูกี้ที่เครื่องไคลเอนต์โดยใช้ HTTP session state

และเมื่อมี SSL เข้ามา เครื่องที่ทำหน้าที่เป็นโหนดบาลานเซอร์จะทำการเข้ารหัสและถอดรหัสระหว่างเครื่องไคลเอนต์และเว็บโลจิกเซิร์ฟเวอร์และโหนดบาลานเซอร์จะใช้ plain text ที่เว็บโลจิก เซิร์ฟเวอร์ ใช้กับเครื่องไคลเอนต์ มาเพื่อควบคุมการติดต่อกันระหว่างเครื่องไคลเอนต์และเครื่องเซิร์ฟเวอร์อื่นๆที่อยู่ในคลัสเตอร์

Proxy Requirements

เว็บโลจิกพรอกซี ทำการติดต่อเพื่อควบคุมการทำงานของ เว็บโลจิก เซิร์ฟเวอร์ แทน ซึ่งเป็นโฮสต์ที่ใช้งานเกี่ยวกับ Servlets และ JSP และทำการส่งข้อมูล HTTP request ไปที่เครื่องเซิร์ฟเวอร์โดยใช้การทำงานแบบ round-robin และพรอกซี ยังทำหน้าที่ในการแบ่งการทำงานที่เครื่องเซิร์ฟเวอร์ส่งไปให้เครื่องไคลเอนต์ได้อีกด้วยถ้า เว็บโลจิก เซิร์ฟเวอร์ เกิดการทำงานผิดพลาด และเว็บโลจิก เซิร์ฟเวอร์ สามารถรองรับการทำงานของเว็บเซิร์ฟเวอร์ และพรอกซีซอร์ฟแวร์ดังต่อไปนี้

- Netscape Enterprise เซิร์ฟเวอร์โดยมี Netscape (proxy) plug-ins
- Apache เซิร์ฟเวอร์โดยมี (proxy) plug-ins
- Microsoft Internet Information เซิร์ฟเวอร์โดยมี Microsoft-IIS (proxy) plug-ins
- เว็บ โลจิกเซิร์ฟเวอร์โดยมี HttpClusterServlet

Configuring IN-Memory HTTP Replication

ทุกเว็บแอปพลิเคชันที่ทำงานบน เว็บ โลจิก เซิร์ฟเวอร์ จะมี เว็บ โลจิก Deployment Descriptor(weblogic.xml) ผู้ใช้ควรที่จะเซตค่าพารามิเตอร์ *PersistentStoreType* ให้ทำการเก็บค่าใน *weblogic.xml* เพื่อเป็นการเปิดใช้งานของ in-memory HTTP session replication

File-Based Replication

File-Based Replication ทำหน้าที่ในการเก็บค่าเซสชันต่างๆในไฟล์

Configuring File-Based Session Persistence

Configuring File-Based Session Persistence สามารถถูกคอนฟิกโดยการใส่ค่าแอตทริบิวต์ 2 ค่าใน เว็บ โลจิก Deployment Descriptor(weblogic.xml) ซึ่งทั้ง 2 ค่านั้นคือ

- ข้อมูลของ *PersistentStoreType* ที่อยู่ในส่วนของ `<session-descriptor>` ซึ่งอยู่ภายในไฟล์
- ข้อมูลของ *PersistentStoreDir* ซึ่งเป็นส่วนที่ควรเก็บไว้ในเซิร์ฟเวอร์

JDBC-Based Session Persistence

JDBC-Based Session Persistence จะเก็บข้อมูลเซสชันต่างๆลงในฐานข้อมูล

กาตั้งค่า JDBC-Based Session Persistence

คือการปรับค่าต่างๆที่อยู่ในไฟล์ *weblogic.xml*

- เซตคุณลักษณะ *PersistentStoreType* ที่อยู่ในส่วน `<session-descriptor>` และ JDBC ที่อยู่ใน เว็บ โลจิก Deployment Descriptor(weblogic.xml)
- เซตการเชื่อมต่อที่ต้องการใช้ (เซตค่าการเชื่อมต่อ JDBC ที่ถูกใช้เพื่อเก็บข้อมูลกับ *PersistentStorePool*)
- เซตค่า Access Control List(ACL) เพื่อการเชื่อมต่อและเช็คการมีสิทธิการใช้งานของผู้ใช้ต่างๆ
- เซตค่าตารางในฐานข้อมูลที่มีชื่อว่า *w1_servlet_session* สำหรับ JDBC-based persistence การเชื่อมต่อนี้จะต่อกับฐานข้อมูลที่ต้องการใช้ในการเขียนหรืออ่านตารางนี้

ออบเจกต์คลัสเตอร์

ออบเจกต์คลัสเตอร์ เป็นออบเจกต์ที่ใช้งานโดยที่เครื่องไคลเอนท์ใช้ Remote Method Invocation(RMI) ยกตัวอย่างเช่น EJB ออบเจกต์คลัสเตอร์สามารถทำงานบน เว็บโลจิกคลัสเตอร์ได้และสามารถทำงานบนเว็บโลจิกเซิร์ฟเวอร์ได้อีกด้วย

เมื่อ EJB ที่รองรับการทำคลัสเตอร์ถูกคอมไพล์ *ejbc*(เป็นคอมไพเลอร์จาก BEA) จะทำการส่งค่าผ่านทาง อินเทอร์เฟซของ EJB ผ่านทาง *rmic*(RMI ออบเจกต์ compiler tool) และคอมไพล์เพื่อที่จะทำเป็นค่าสแต็บเพื่อนำมาใช้ใน EJB replica-aware stub เป็นสแต็บที่ใช้งานกับเครื่องไคลเอนท์ของ EJB แต่ replica-aware stub บรรจุการทำงานของ EJB หรือ RMI ที่อยู่บน เว็บโลจิก เซิร์ฟเวอร์ เมื่อเครื่องไคลเอนท์ทำการติดต่อกับคลัสเตอร์ออบเจกต์ แล้ว replica-aware stub จะถูกส่งไปที่เครื่องไคลเอนท์ โดยสแต็บอันนี้จะบรรจุฟังก์ชันการทำงานของโหนดบาลานซ์ซึ่งเพื่อใช้ในการเรียกออบเจกต์ และในแต่ละครั้งสแต็บจะใช้ฟังก์ชันโหนดบาลานซ์ของตนเองเพื่อเลือกว่าตัวไหนจะถูกเรียก แล้วก็จะทำการส่งไปยังเครื่องไคลเอนท์ และในกรณีที่ไม่สามารถทำงานได้สแต็บจะตัดการทำงานแล้วพยายามติดต่อกับอีกเซิร์ฟเวอร์เพื่อทำการ เฟลโอเวอร์

คลัสเตอร์ EJBs

EJB สามารถที่จะมี replica-aware stub ได้ 2 อย่างเพื่อที่จะทำเป็น EJBHome และ EJBObject ดังนั้น EJB สามารถรู้ถึงผลที่เกิดของโหนดบาลานซ์และการเฟลโอเวอร์ ใน 2 ระดับคือ

- client look-up time
- client method invocation time

เมื่อ EJB ทำงานบนเซิร์ฟเวอร์ เซิร์ฟเวอร์นั้นจะถูกล้อมรอบไปด้วย Cluster-wide naming เซอร์วิส เพราะว่า EJB home สามารถทำการคลัสเตอร์ได้ และแต่ละเซิร์ฟเวอร์นำมารวมกันโดยใช้ชื่อเดียวกันได้เลย และเมื่อเครื่องไคลเอนท์มองหา มันจะได้ค่าที่เป็น replica-aware stub ซึ่งอ้างอิงกับข้อมูลที่อยู่จริงได้ และเมื่อ home ต้องการที่จะหาหรือสร้าง bean มันจะต้องสร้างขึ้นมานบนเครื่อง local เซิร์ฟเวอร์ และส่งค่าสแต็บคืนมายังที่เซิร์ฟเวอร์เดิมและทำการ เฟลโอเวอร์ เฉพาะที่ระดับเดิมเท่านั้น เพราะว่ามันเป็นไปไม่ได้ที่จะทำทั้งหมดเนื่องจากค่า instance จะต้องอ่านค่าจากฐานข้อมูลก่อนการส่งจะเริ่มต้นขึ้นและก่อนการ commit

และเมื่อ home ค้นหาหรือสร้าง bean ได้แล้ว มันจะส่งค่า replica-aware stub ซึ่ง stub load balance ของทุกการเรียกนั้นไม่สามารถทำการแก้ไขการเฟลโอเวอร์อย่างอัตโนมัติทุกตัว